

InfoDays: Software-Architektur · 07.10.2026

STACKIT im Praxistest

Deployment, Developer Experience und Architektur-Fit

Florian Lenz · Cloud Architekt

Die eine Frage hinter diesem Vortrag

**Kann ich meine nächste Anwendung heute auf
STACKIT betreiben und wie fühlt sich das an?**

Wie gut ist die Developer Experience?

Wo gibt es noch Einschränkungen?

Welche Services sind produktionsreif?

Für welche Workloads passt es heute?

Ablauf

1

Architektur

Was eine typische App braucht und was STACKIT davon abdeckt

2

Live-Demo

Terraform, Forgejo-Pipeline, App im Publikum

3

Bewertung

Reifegrad pro Service, Stolpersteine, Zahlen

4

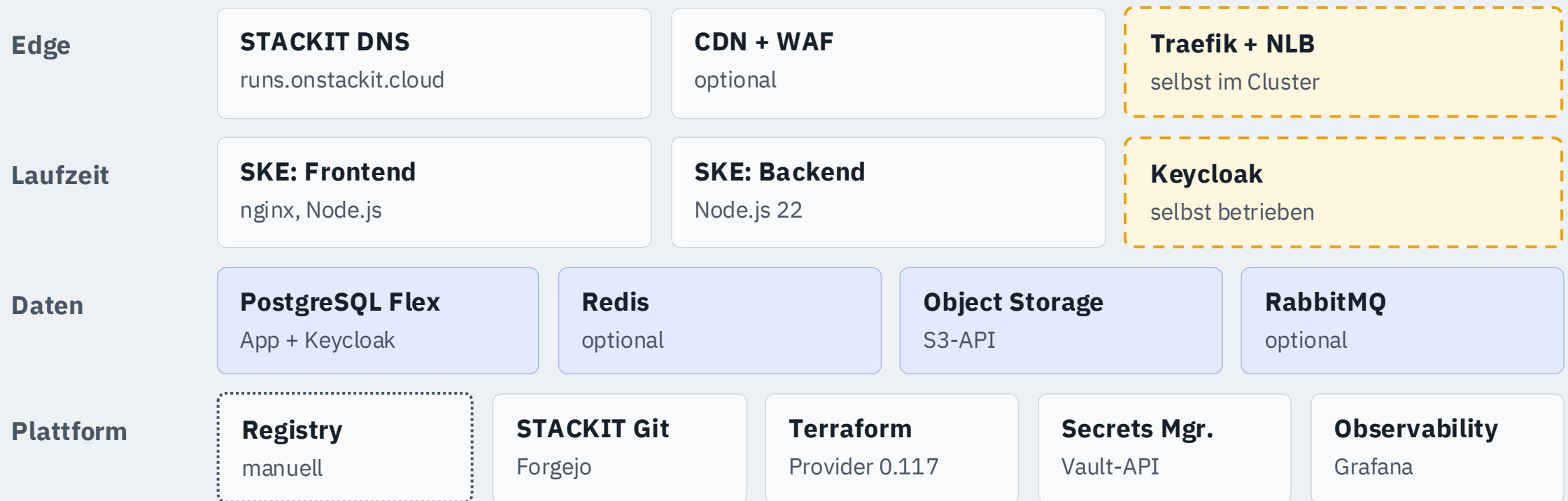
Einordnung

Vergleich mit Azure und passende Workloads

Was eine Anwendung heute braucht

Edge	DNS & TLS Domain, Zertifikate	CDN / WAF optional	Load Balancer Ingress, Routing		
Laufzeit	Frontend Container	Backend-API Container	Identity Provider Login, Rollen		
Daten	Datenbank managed	Cache optional	Object Storage Dateien	Queue optional	
Plattform	Registry Images	CI/CD Build, Deploy	IaC Terraform	Secrets Vault	Observability Logs, Metriken

Dieselbe Architektur mit STACKIT-Diensten



Die Demo-App: askit

Ihr stellt Fragen zum Vortrag und stimmt ab. Ich markiere Beantwortetes und lasse die offenen Fragen per KI zusammenfassen.

Klein, aber mit allem, was eine echte App braucht: Login, Datenbank, Datei-Upload, KI-Aufruf, Metriken.

[QR-Code zur App]

[app.....runs.onstackit.cloud]

Teil 2

Live-Demo

Vom leeren Projekt bis zur App im Publikum

Akt 1: Infrastruktur in zwei Terraform-Stacks

```
resource "stackit_ske_cluster" "main" {  
  name = "askit"  
  extensions = {  
    dns = {  
      enabled = true  
      zones  = [stackit_dns_zone.main.dns_name]  
    }  
    observability = {  
      enabled    = true  
      instance_id = stackit_observability_instance...  
    }  
  }  
}
```

10-cloud

SKE, PostgreSQL Flex, Object Storage, DNS, Secrets Manager, Observability, Git, KI-Token

20-platform

Traefik, cert-manager, External Secrets, Secrets befüllen, CI-Zugang

Was nicht per Terraform ging

Container Registry

Harbor unter registry.onstackit.cloud. Projekt und Robot Accounts im Portal, das Token wird nur einmal angezeigt.

Git-Runner

Managed Runner werden im Portal aktiviert. Die Instanz selbst ist in Terraform eine Beta-Ressource.

ALB für SKE

Die Erweiterung ist Private Preview. Deshalb Traefik plus automatisch erzeugter Network Load Balancer.

Plattform-Arbeit, die ich selbst übernehme

Ingress

Traefik per Helm, NLB entsteht automatisch aus dem Service.

Zertifikate

cert-manager mit Let's Encrypt, HTTP-01 über Traefik.

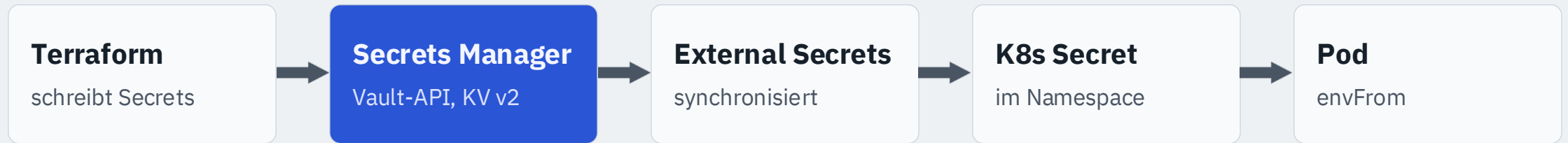
Secret-Sync

External Secrets Operator liest den Secrets Manager über die Vault-API.

Identity

Keycloak mit eigener Datenbank, Realm-Import beim Start.

Secrets: ein Weg, keine Kopien



Weil der Secrets Manager die Vault-API spricht, funktionieren Standardwerkzeuge: der Vault-Provider in Terraform, der External Secrets Operator im Cluster und die Vault-Action in der Pipeline.

Akt 2: Vom Commit in die Produktion

```
# .forgejo/workflows/deploy.yaml
jobs:
  build:
    runs-on: stackit-docker
    steps:
      - uses: https://github.com/actions/checkout@v4
      - run: docker login registry.onstackit.cloud ...
      - run: docker build ... && docker push ...
  deploy:
    needs: build
    steps:
      - run: kustomize edit set image ...
      - run: kubectl rollout status ...
```

Unterschiede zu GitHub Actions

- Ordner .forgejo/workflows
- Actions per voller URL
- Runner-Labels von STACKIT
- Ähnlich, aber nicht identisch

Akt 3: KI ohne neues SDK

```
const res = await fetch(`${baseUrl}/chat/completions`, {  
  method: "POST",  
  headers: { authorization: `Bearer ${apiKey}` },  
  body: JSON.stringify({ model, messages }),  
});  
  
// baseUrl = https://api.openai-compat  
// .model-serving.eu01.onstackit.cloud/v1
```

OpenAI-kompatible API mit Open-Weight-Modellen aus deutschen Rechenzentren.

In Terraform nur ein Token. Aber: Modelle werden ersetzt, Migrationsfristen im Blick behalten.

Keycloak: eine Box im Diagramm, ein Produkt im Betrieb

Thema	In der Demo	Für Produktion
Verfügbarkeit	1 Replika	2+ Replikas, PDB
Updates	Version gepinnt	Staging-Upgrade, Release Notes
Admin-Konsole	öffentlich	nur intern erreichbar
E-Mail	keine	SMTP für Reset, Verifizierung
Konfiguration	Realm-JSON beim Start	config-cli oder Terraform

Die Demo in Zahlen

29

Dateien für Infrastruktur, Manifeste und Pipelines

90 min

terraform apply, leeres Projekt bis Cluster

10 min

vom Commit bis zur neuen Version live

Reifegrad pro Service

Service	Einschätzung	Warum
SKE (Kubernetes)	Produktionsreif	GA, Terraform, DNS- und Observability-Extension
PostgreSQL Flex	Produktionsreif	Terraform inkl. User, DB, ACL aus Cluster-IPs
Object Storage	Produktionsreif	S3-API, Standard-SDKs funktionieren
DNS	Produktionsreif	kostenlose Subdomains, ExternalDNS integriert
Secrets Manager	Produktionsreif	Vault-API, ESO und Vault-Provider
AI Model Serving	Produktionsreif	OpenAI-kompatibel, Modell-Lebenszyklus beachten
Container Registry	Einschränkung	Harbor solide, aber kein Terraform
Git + Pipelines	Einschränkung	Beta in Terraform, Actions nicht 1:1
ALB in SKE	Einschränkung	Private Preview, Ingress selbst betreiben
Serverless Container, IdP	Fehlt	kein Pendant zu Container Apps oder Entra

Einordnung gegenüber Azure

Baustein	Azure	STACKIT
Container	Container Apps, AKS	SKE, Cloud Foundry
PostgreSQL	Flexible Server	PostgreSQL Flex
Dateien	Blob Storage	Object Storage (S3-API)
Identity	Entra ID, External ID	selbst: Keycloak
Secrets	Key Vault	Secrets Manager (Vault-API)
CI/CD	GitHub, Azure DevOps	STACKIT Git (Forgejo)
KI	Azure OpenAI, AI Foundry	AI Model Serving, Open-Weight
IaC	azurerm, sehr breit	stackit 0.x, wächst schnell

Für welche Workloads STACKIT heute passt

Passt gut

Containerisierte Web-Apps und APIs mit Kubernetes-Erfahrung im Team, Datenhaltung mit Souveränitätsanforderung, KI mit Open-Weight-Modellen.

Mit Aufwand

Teams ohne Kubernetes-Know-how, Apps mit komplexem Identity-Bedarf, voll automatisierte Plattformen, die Registry und Runner per Code wollen.

Noch abwarten

Serverless-first-Architekturen, die auf Scale-to-zero-Container oder einen gemanagten IdP angewiesen sind.

Fazit

Die Bausteine sind da und sprechen offene Standards. Den Komfort einer Plattform darüber baut man heute noch selbst.

Wer Kubernetes beherrscht, kann produktiv starten. Wer Serverless erwartet, sollte die Roadmap verfolgen.

Eure Fragen

Code, Terraform und Pipelines zum Nachbauen: [Repository-Link]

[Dein Name] · [Kontakt]



[QR-Code Repo]